EVIDENCE LEDGER & AUDIT TRAILS

Aegis Agent / Engineering implementation note / Architect: Amin Azimi

01 | My design intent

I architect Aegis Agent around inspectable execution evidence rather than treating a successful agent response as proof of accountability. I use a local SHA-256 hash chain to make changes to stored event content detectable when the chain is subsequently verified. This is a tamper-evidence mechanism, not a claim that a local database is immutable or externally notarized.

02 | What I implemented

Mechanism	Observed implementation
Evidence record	Index, timestamp, tenant ID, actor, action, payload, previous hash and current hash.
Canonical digest	SHA-256 of sorted, compact JSON containing the event fields and previous hash.
Persistence	SQLite evidence_ledger table; rows reload into the ledger when an instance starts.
Verification	verify_chain() checks each digest and the link to the preceding record.
Inspection	snapshot() exposes entries; find_tamper_indices() reports inconsistent positions.

03 | Evidence flow

Event → canonical JSON → SHA-256 digest → previous-hash linkage → SQLite row → later verification. I preserve tenant identifiers as fields on events, while the chain itself is global to the ledger instance rather than a separate chain per tenant.

04 | Practical engineering value

I can reconstruct the sequence of recorded actions and detect mismatches in their stored hash linkage. This supports troubleshooting, audit review and investigation of accidental or deliberate edits to existing rows, provided the verifier and its reference data remain trustworthy.

AUDITABILITY, LIMITS & SOURCE EVIDENCE

05 | What the code does not guarantee

Not append-only: I explicitly distinguish a hash-linked history from a write-once audit system. The current module exposes remove_last(), which deletes trailing entries from memory and SQLite. SQLite rows can also be modified by an actor with sufficient database access.

No external anchor: I do not claim independent timestamp attestation, remote notarization, a hardware-backed signing key, or cryptographic proof against an administrator who can rewrite the entire chain.

Persistence failure boundary: append_entry() updates the in-memory list before writing the SQLite row. If persistence fails, in-memory and durable state can diverge. I do not describe this path as transactional end-to-end.

Coverage boundary: Some caller paths use best-effort evidence logging and suppress logging exceptions. I therefore do not claim that every operational event is durably captured. Nor do I equate a stored tenant_id field with independently verified tenant authorization.

06 | Example verification logic

For each stored entry, I recompute SHA-256 over the canonical event fields, compare the result with its recorded hash, and verify that its prev_hash matches the preceding record. A mismatch flags inconsistency; a clean check establishes only internal chain consistency for the data currently presented to the verifier.

07 | Related observability, kept distinct

I maintain a separate in-memory Scorecard for counters such as approvals required, rollbacks, signature totals and sandbox passes. I do not present those counters as durable evidence-ledger records, nor as measured production reliability statistics.

08 | Source traceability

- core/evidence_ledger.py — LedgerEntry, append_entry(), verify_chain(), find_tamper_indices(), remove_last().
- core/persistence.py — SQLite database location, connection settings and WAL configuration.
- core/scorecard.py — in-memory counters; separate from durable evidence records.
- core/model_router.py and core/token_optimizer.py — examples of evidence-logging call sites.

09 | My engineering position

I present the ledger as an implemented, locally persisted tamper-evident event chain with explicit operational limitations. I do not market it as immutable storage, a compliance certification, a complete security audit, or proof of test results that were not independently executed.

Repository: github.com/aminazimi42-coder/aegis-agent | Source-level review; no runtime test execution claimed.